엔지니어링으로서의 설계 결정과 트레이드오프

lean storage, hybrid search,
schema-driven extensibility

SECTION OVERVIEW

11개 아키텍처 관점

Facet 0 — 아키텍처 개요: lean storage + rich ontology

Facet 1 — 하이브리드 검색: 세 가지 실패 모드 방지

Facet 2 — Schema-driven extensibility

Facet 3 — 데이터 모델: 노드 스키마의 설계

Facet 4 — 세션 주입 파이프라인

Facet 5 — 온톨로지 건강 관리

Facet 6 — MCP 서버 아키텍처

Facet 7 — 에코시스템 토폴로지

Facet 8 — Context as Currency 구현

Facet 9 — 기술 부채와 알려진 제한

Facet 10 — 아키텍처 결정의 원칙

FACET 00

아키텍처 개요: lean storage + rich ontology

mcp-memory의 물리 저장은 2테이블 구조다 — nodes와 edges. 이 단순함이 의도적이다. 저장 엔진을 무겁게 만들지 않고, 의미 체계는 schema.yaml로 외부화한다. "이 파일을 편집하면 코드 변경 없이 타입 확장 가능."

> 관심사 분리(separation of concerns)의 교과서적 적용 — 저장(how)과 의미(what)를 분리.

이 선택의 트레이드오프: 저장 엔진이 타입 제약을 강제하지 않는다. validators.py가 런타임에서 검증하지만, DB 수준의 제약(foreign key, check constraint)은 없다. 유연성을 얻고 안전성을 일부 포기했다.

FACET 01

하이브리드 검색: 세 가지 실패 모드 방지

SQLite FTS + ChromaDB + Graph traversal. 각각이 다른 실패를 막는다.

SQLite FTS: 정확한 키워드 매칭. "Phase v3"를 찾을 때 벡터 검색은 "Phase v2"도 가져올 수 있지만, FTS는 정확히 "Phase v3"만 잡는다.

ChromaDB: 의미적 유사성. "토큰 절약"과 "컨텍스트 최적화"는 다른 표현이지만 같은 의미. FTS로는 못 잡고 벡터 검색으로 잡는다.

Graph traversal: 명시적 관계 복구. "이 Failure가 어떤 Decision에서 나왔는가?"는 유사도가 아니라 edge를 따라가야 한다.

> 세 엔진의 결과를 병합하는 전략이 retrieval 품질의 핵심이다.

현재 NDCG 측정 스크립트가 v1.0으로 완료된 상태이고, re-embed를 통한 NDCG 0.9 목표가 다음 마일스톤이다.

FACET 02

Schema-driven extensibility

schema.yaml이 런타임에 가까운 설계 원본이라는 점이 핵심 아키텍처 결정이다. 코드가 schema를 읽고, 타입과 관계를 동적으로 인식한다. 새 타입을 추가하려면 YAML에 한 줄 추가하면 된다.

이 패턴의 장점: (1) 비개발자도 온톨로지를 수정할 수 있다 (2) 타입 변경이 코드 변경을 요구하지 않는다 (3) schema의 버전 관리가 곧 온톨로지의 진화 이력이 된다.

> 단점: schema와 코드의 동기화가 런타임에서만 검증된다. Compile-time safety가 없다. 잘못된 타입명을 넣어도 YAML 파싱까지는 에러가 안 난다.

FACET 03

데이터 모델: 노드 스키마의 설계

노드 스키마의 필드: type, content, metadata, project, tags, confidence, source, created, updated. 각 필드가 retrieval과 reasoning을 위해 설계되어 있다.

confidence: 지식의 확실성. 승격 게이트의 입력. source: 출처 추적. 할루시네이션 방지. project: 적용 범위. 크로스 프로젝트 검색 vs 프로젝트 내 검색 분리. tags: 비구조적 메타데이터. 온톨로지 타입으로 포착 안 되는 속성.

> 이 필드들은 "보조 정보"가 아니다. 기억의 해석 가능성을 만드는 핵심이다.

content 없이 type + confidence + project만으로도 "이 노드가 어떤 성격의 지식인지"를 알 수 있다.

FACET 04

세션 주입 파이프라인

session_context.py가 세션 시작 시 수행하는 작업: (1) L2+ 노드를 confidence 순으로 정렬, 상위 15개 선택 (2) 최근 30일 Signal 타입 추가 (3) 최근 7일 Observation 추가 (4) Correction 타입 전량 추가 (5) 전체를 토큰 예산 내로 조정.

> retrieval과 injection이 분리되어 있다. 검색은 "무엇이 관련 있는가"를 찾고, 주입은 "무엇을 의식에 올릴 것인가"를 결정한다.

이 파이프라인의 아키텍처적 의미: 같은 데이터에 대해 다른 정책을 적용할 수 있다.

FACET 05

온톨로지 건강 관리

ontology_review.py와 ontology_health.py가 온톨로지의 "건강"을 관리한다. 측정 항목: 고아 노드(edge 없는 노드), 타입 분포 편향, 오래된 저confidence 노드, 관계 밀도.

Phase 6에서 "orphan 571개 연결" 작업이 수행되었다. 이건 기술 부채 청산이다. 노드만 만들고 관계를 안 맺으면 그래프가 아니라 리스트가 된다. 정기적 건강 관리가 그래프의 가치를 유지한다.

> auto-iterate의 3-way 스캔: (1) 타입 분포 (2) 레이어 분포 (3) 관계 밀도. 임계값을 벗어나면 자동으로 교정 행동을 트리거한다.

dry_run=false로 전환된 상태 — 비파괴적 자동 행동이 활성화되어 있다.

FACET 06

MCP 서버 아키텍처

mcp-memory는 MCP(Model Context Protocol) 서버로 구현되어 있다. 이 선택의 의미: Claude Code, 다른 LLM 클라이언트, 심지어 GPT에서도 접근할 수 있다. 특정 LLM에 종속되지 않는 인터페이스.

MCP의 장점: 표준화된 도구 호출 프로토콜. 단점: MCP 생태계가 아직 초기여서 클라이언트별 호환성 이슈가 있다. context export MCP 도구가 4개 플랫폼에서 동작 확인된 상태.

> LLM 비종속성이 다른 모든 트레이드오프보다 중요하다. 사용자의 인지 구조가 특정 LLM 벤더에 종속되면 안 된다는 설계 원칙.

그러나 MCP 생태계는 아직 초기다 — 클라이언트별 호환성 이슈가 존재한다. 비동기 처리가 프로토콜 수준에서 제한적이어서, 대량 노드 일괄 처리 시 성능 병목이 될 수 있다. SSE 기반 통신은 WebSocket보다 양방향성이 약하고, 클라이언트 재연결 시 상태 복구가 복잡하다. MCP 표준 자체가 아직 진화 중이어서 breaking change 위험이 있다.

FACET 07

에코시스템 토폴로지

12개 프로젝트의 의존성 그래프: orchestration이 허브, 나머지가 스포크. 이건 의도적인 star topology다. 모든 프로젝트가 서로 직접 연결되면 $O(n^2)$ 복잡성. 중앙 orchestration을 통한 간접 연결은 $O(n)$.

> index-system이 이 토폴로지를 자동 추적한다. 25 nodes, 539 edges, 38.7M tokens — 이것이 에코시스템의 현재 스냅샷.

git-remote, syncs-to, delegates-to, skill-of 4종 edge가 프로젝트 간 관계를 추적한다.

FACET 08

Context as Currency 구현

200K 컨텍스트 관리의 구체적 수치: 42K baseline (CLAUDE.md + rules + 기본 상태). 70K 경고 (비필수 컨텍스트 오프로딩 시작). 120K 주의 (새 에이전트 호출 제한). 150K 자동 압축 (auto-compact 트리거). 200K 하드 리밋.

> 체인 결과 오프로딩 패턴: 에이전트 체인의 중간 결과를 .chain-temp/에 파일로 저장하고, 컨텍스트에서는 요약만 남긴다. 이것으로 20-30K 토큰 절감.

FACET 09

기술 부채와 알려진 제한

현재 식별된 기술 부채: validators.py의 타입 전이 규칙이 얼마나 강하게 적용되는지 불확실. relation별 source/target 제약이 코드에서 완전하지 않음. materialized 25개 타입 vs schema 51개 타입의 간극. re-embed 작업 미완료 (NDCG 0.9 목표 미달).

> SQLite는 단일 사용자, 단일 프로세스에 최적화. 멀티 에이전트가 동시 쓰기하면 충돌.

"에이전트 4개가 같은 JSON 동시 수정 시 충돌" 경험. 해결: 쓰기 직렬화 또는 WAL 모드. 근본 해결은 아님.

FACET 10

아키텍처 결정의 원칙

mcp-memory의 아키텍처 결정들을 관통하는 원칙:

1. 단순한 저장, 풍부한 의미. 물리 구조는 2테이블. 의미 구조는 51 타입, 50 관계.

2. 외부화된 스키마. 코드에 타입을 하드코딩하지 않는다. YAML이 진실의 원천.

3. 다중 검색 전략. 하나의 검색 방법으로는 모든 실패를 막을 수 없다.

4. 측정 가능한 건강. 시스템의 상태를 수치로 볼 수 있어야 교정할 수 있다.

5. 점진적 복잡성. 5개 타입으로 시작해서 25개로 자란다. 51개 자리는 예비되어 있지만 채우지 않는다.

> lean storage의 대가로 ACID 트랜잭션이 약하다. schema-driven 접근의 런타임 검증은 서버 시작 시에만 수행된다. 단일 사용자 SQLite는 스케일링 천장이 있다.

노드 생성과 엣지 생성이 원자적이지 않아 orphan이 발생할 수 있다. 운영 중 schema 수정의 안전성이 보장되지 않는다. 노드 10만 개 이상에서의 검색 성능이 미검증이다.

COLOPHON

Lens C — 소프트웨어 아키텍처
mcp-memory System Analysis, 2026-03-28

Paul Park × Claude

lean storage, hybrid search, schema-driven extensibility
MCP protocol, context engineering, ecosystem topology
11 Facets · 5 architecture principles

Typeset in Inter + Noto Serif KR
247 × 340 mm · El Croquis Design System v2.0